

Estrutura de Dados

Inteligência Artificial e Ciência de Dados | 2026/1



Prof Vitor

Unisenac
Centro Universitário RS



Fecomércio
IFEP
Sesc
Sindicatos Empresariais



O nosso caso

Vamos trabalhar com uma pequena base de vendas, na qual cada registro possui **id**, **cliente**, **categoria**, **cidade** e **valor**.

```
vendas = [  
  {"id": 101, "cliente": "Ana", "categoria": "Eletrônicos", "cidade": "Porto Alegre", "valor": 3500},  
  {"id": 102, "cliente": "Bruno", "categoria": "Livros", "cidade": "Canoas", "valor": 80},  
  {"id": 103, "cliente": "Carla", "categoria": "Eletrônicos", "cidade": "Porto Alegre", "valor": 1200},  
  {"id": 104, "cliente": "Diego", "categoria": "Casa", "cidade": "São Leopoldo", "valor": 230},  
  {"id": 105, "cliente": "Eva", "categoria": "Livros", "cidade": "Canoas", "valor": 95}  
]
```

O problema

Queremos responder perguntas como:

- Onde está a venda de ID 103?
- Como recuperar esse registro?

```
vendas = [  
  {"id": 101, "cliente": "Ana", "categoria": "Eletrônicos", "cidade": "Porto Alegre", "valor": 3500},  
  {"id": 102, "cliente": "Bruno", "categoria": "Livros", "cidade": "Canoas", "valor": 80},  
  {"id": 103, "cliente": "Carla", "categoria": "Eletrônicos", "cidade": "Porto Alegre", "valor": 1200},  
  {"id": 104, "cliente": "Diego", "categoria": "Casa", "cidade": "São Leopoldo", "valor": 230},  
  {"id": 105, "cliente": "Eva", "categoria": "Livros", "cidade": "Canoas", "valor": 95}  
]
```



Busca linear

Na busca linear:

- percorremos os registros um a um
- comparamos o id desejado com o id de cada item
- paramos quando encontramos

```
def busca_venda_por_id(vendas, id):  
    //vamos implementar?
```



Busca linear

Na busca linear:

- percorremos os registros um a um
- comparamos o id desejado com o id de cada item
- paramos quando encontramos

```
def busca_venda_por_id(vendas, id):  
    for venda in vendas:  
        if venda["id"] == id:  
            return venda  
    return None
```


Por que isso pode ser um problema?

Se a base crescer:

- a busca pode ficar lenta
- o custo cresce com o número de elementos
- no pior caso, precisamos olhar tudo e temos uma **complexidade $O(n)$**

A *tabela hash*

Uma **tabela *hash*** é uma estrutura que:

- recebe uma chave
- calcula uma posição
- guarda ou recupera o valor a partir dessa posição

A ideia central é não percorrer tudo e tentar **chegar direto ao lugar** do dado.

Chave & Valor

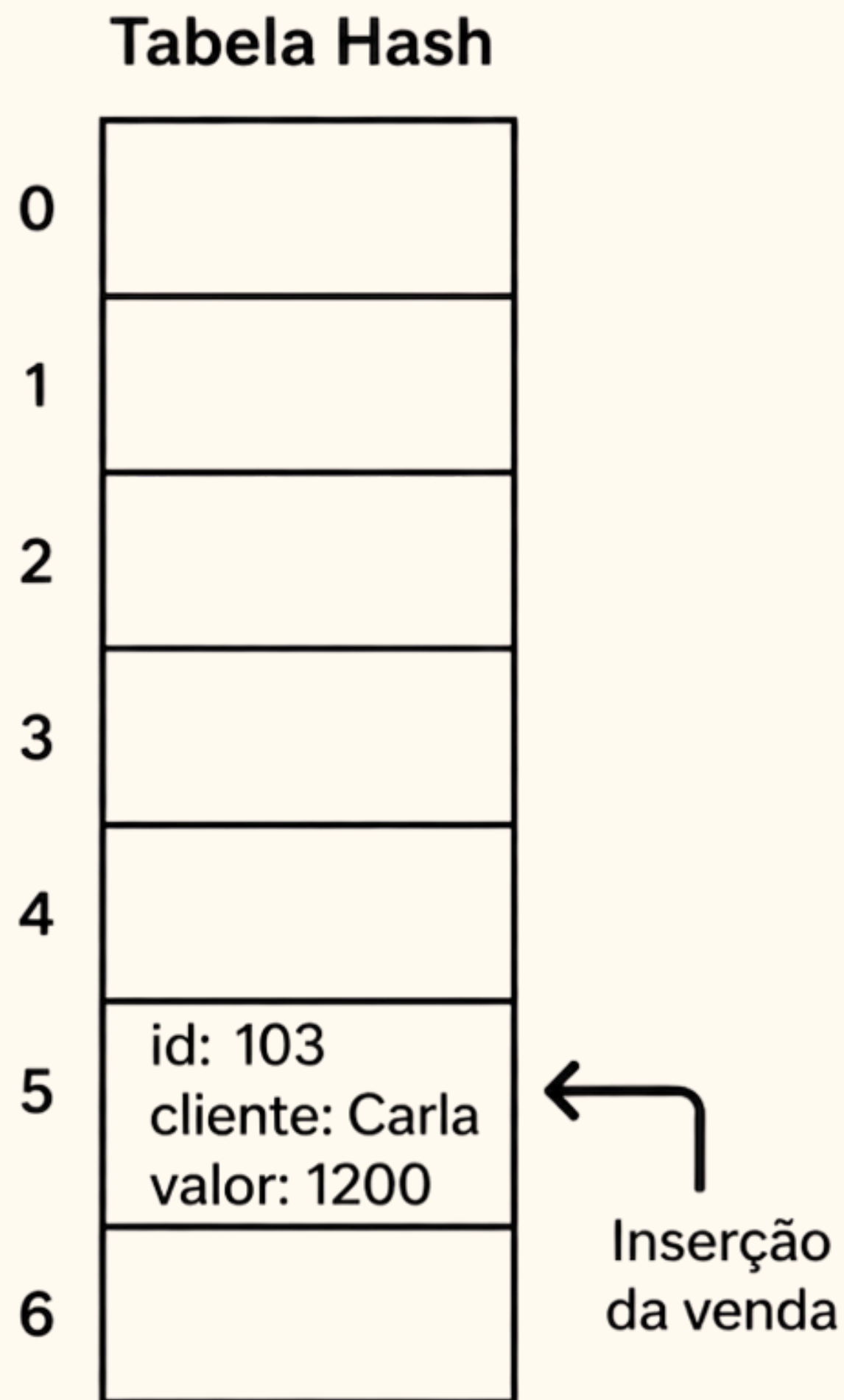
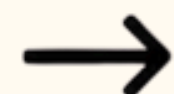
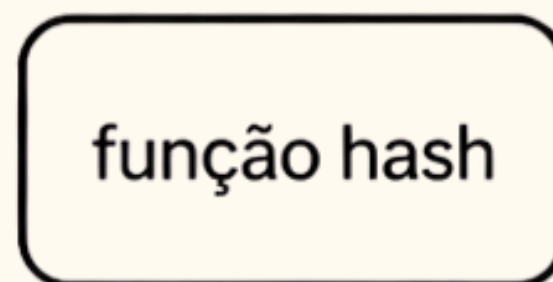
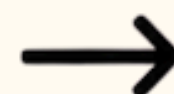
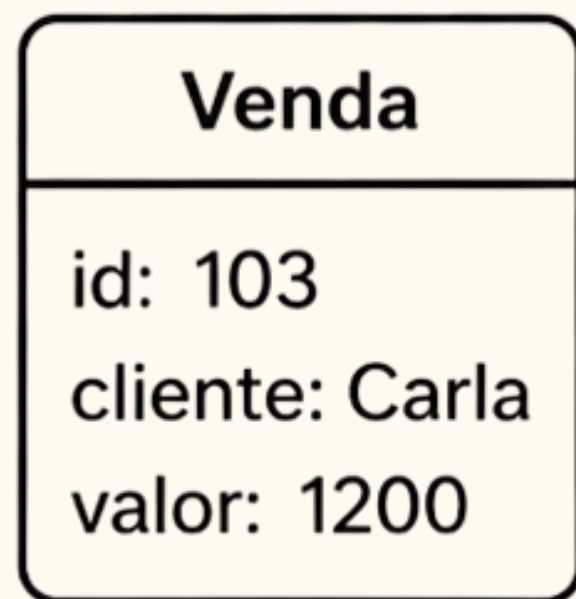
Numa tabela hash, normalmente trabalhamos com:

- uma chave
- um valor associado

No nosso exemplo:

- chave: id
- valor: registro da venda

```
103 -> {"id": 103,  
        "cliente": "Carla",  
        ...}
```

A função hash

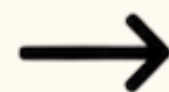
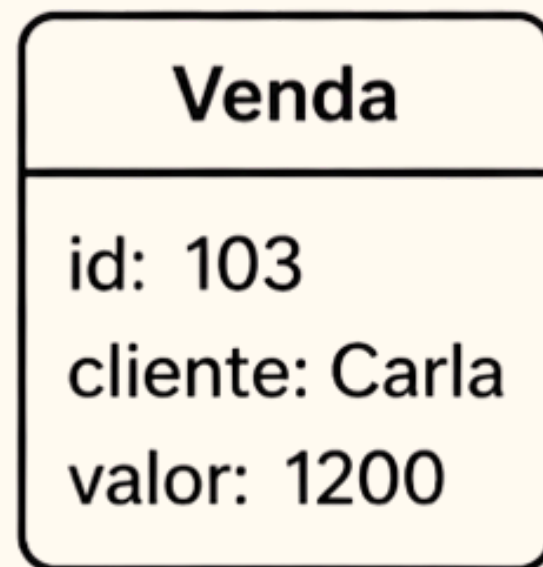
- Uma função hash é uma função que:
 - recebe uma chave
 - transforma essa chave em um índice
 - ajuda a decidir onde o dado será armazenado ou procurado

As várias funções hash

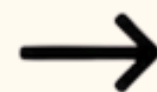
- Existem diversas abordagens para construir funções hash.
- A nossa busca é sempre por uma função hash que evite **colisões**.
- Uma colisão significa gerar um **índice** (uma posição) **igual para duas chaves diferentes**.

Uma função simples

- Uma implementação extremamente simples de uma função hash, é utilizar o **mod (%) pelo tamanho da lista**.
 - Se utilizar string como chave, utilize o tamanho da string mod o tamanho da lista.
 - Se for um número, como no exemplo, utilize o número mod tamanho da lista.



função hash



$$103 \% 7 = 5$$

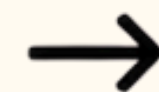


Tabela Hash

0	
1	
2	
3	
4	
5	id: 103 cliente: Carla valor: 1200
6	

←
Inserção
da venda

E a colisão?

- Eventualmente irá acontecer. Por exemplo, em uma lista de tamanho 7, uma chave 101 ou 108 resultariam a mesma coisa [índice 3].
- Algumas soluções são:
 - **Encadeamento**
 - Sondagem linear
 - Sondagem quadrática
 - Duplo *hash*, etc

Encadeamento

- Em vez de cada posição guardar um único valor, cada posição guarda uma lista de itens.
- Então, se houver colisão:
 - os dois elementos ficam no mesmo índice
 - dentro de uma lista
- Isso mata um pouco a complexidade $O(1)$, mas ainda garante performance.



Vamos implementar?

Obrigado!



Quer trocar mais uma ideia? Me chama!

vhlopes@senacrs.com.br